CBSE SCIENCE EXHIBITION

SAHAYAK

REAL-TIME INDIAN SIGN LANGUAGE RECOGNITION SYSTEM



Complete explanation from zero

Architecture, diagrams, code and data flow

How to add signs and train with GitHub

Demo script, roles and judge Q&A

Prepared for the CBSE Science Exhibition

Developer: Mr. Johan Jyoti Kuli

Team: Shyamantika Pegu | Barakha Doley | Himanghsu Ghirme

Study guide + presentation notes + viva preparation

Repository: Mrjohanjyotikuli/SAHAYAK-ISL

Prepared 19 October 2026

Honest scope: limited trained-sign prototype, not a complete ISL translator.

How to use this guide

This guide reflects the SAHAYAK-ISL repository and deployment configuration checked in October 2026. Read the architecture and sign-training chapters before rehearsing the demonstration.

Three rules for the exhibition

- Describe SAHAYAK as a prototype that recognizes a limited set of trained signs, not a complete ISL sentence translator.
- Use verified figures only. The repo does not include a held-out accuracy report or original single-sign training-row count.
- Clearly separate existing features from planned work; reusable collection and training scripts are missing in this repo snapshot.

Quick facts

Area	Verified fact
Frontend	Static HTML/CSS/JavaScript site on Vercel.
Live Studio	Browser camera, MediaPipe Hands JS, local landmark drawing, static/motion prediction.
Backend	Python Flask API hosted on Render; Joblib loads model artifacts.
Static features	126 numeric values; 63 values per hand slot.
Sequence features	132 values per frame; ten time bins form a 1,320-value vector.
Configured labels	HELLO, I_LOVE_YOU, NO, THANK_YOU, YES; /api/status loaded labels are authoritative.
Training	Dedicated static capture and reusable model trainers are not included in current repo snapshot.

Safe description

The project demonstrates hand-landmark tracking, trained sign classification, text output and optional speech/translation.
It does not claim perfect recognition, full ISL understanding or benchmarked accuracy.

1. The problem and why it matters

Spoken communication is not accessible to everyone. Deaf and hard-of-hearing people may use sign languages, writing, interpreters and other methods. SAHAYAK demonstrates one technical pathway: recognize a limited set of trained signs and make the labels available as text or speech.

Published context - not ISL-user counts

Figure	What it measures	Source
5,071,007	India Census 2011 count in the 'disability in hearing' category. Historical, not current estimate or ISL-signer count.	Census C-20, 2011.
430 million	People worldwide requiring rehabilitation for disabling hearing loss, including children.	WHO, 3 Mar 2026.
95.1 million	Children aged 5-19 living with hearing loss worldwide.	WHO, 3 Mar 2026.
2.5 billion by 2050	WHO projection for people with some degree of hearing loss.	WHO, 3 Mar 2026.

Important wording

Hearing loss is not the same as using sign language. These figures describe hearing loss, not the number of ISL users.

Census of India, Table C-20: https://censusindia.gov.in/nada/index.php/catalog/43369/

WHO, Deafness and hearing loss: https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss

2. Objectives, scope and limitations

Objectives

- Demonstrate camera-based hand tracking without special sensor gloves.
- Show separate classification paths for static poses and motion sequences.
- Overlay landmarks and display recognized labels.
- Allow the message to be translated online and spoken through device/server speech.
- Host a website that opens on a phone or computer.

Current limitations

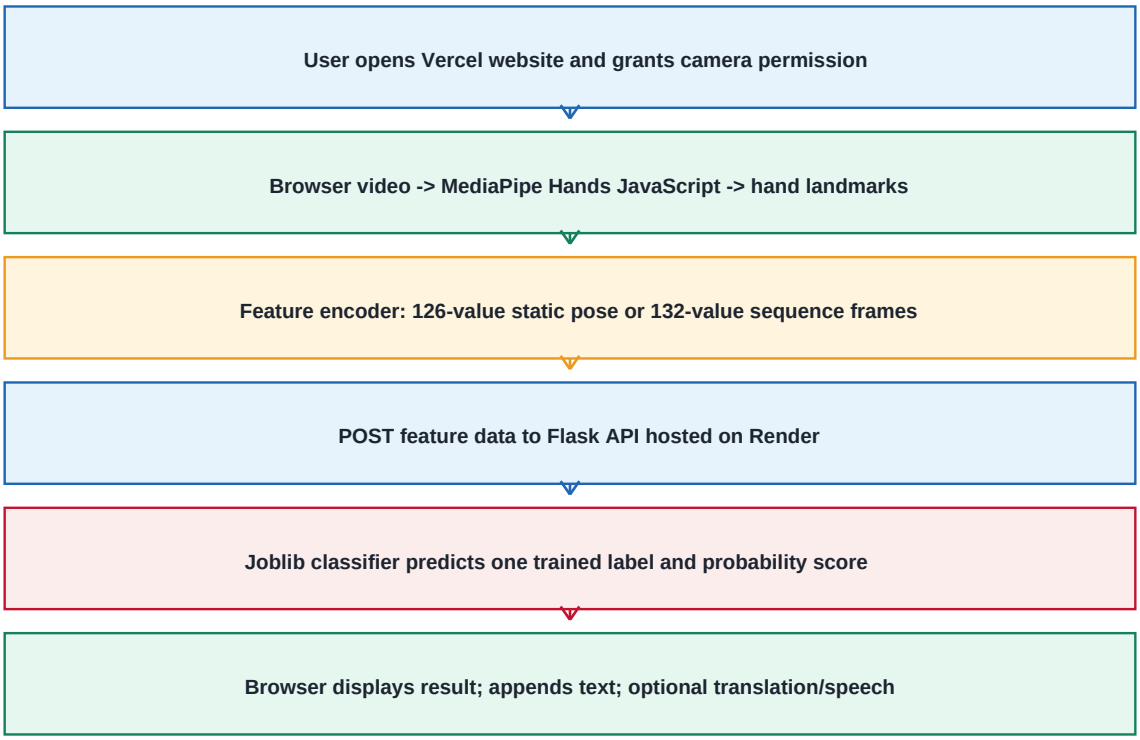
- The model recognizes only classes in its trained artifact; a phrasebook card does not train the model.
- It does not fully interpret ISL grammar, facial expressions, spatial grammar or spontaneous sentences.
- No held-out accuracy report or original static training-row total is stored in the repo.
- The reusable train_twohand_model.py and train_sequence_model.py scripts and a static capture utility are absent.
- SQLite sample writes on a service filesystem are not guaranteed durable across instance replacement.
- MediaPipe loads from a CDN; classification, general translation and gTTS speech use remote services.

Opening line

SAHAYAK is a real-time sign-recognition prototype that converts a limited set of trained hand poses or movements into labels, text and optional speech. It is not a replacement for fluent ISL users or interpreters.

3. System architecture diagram

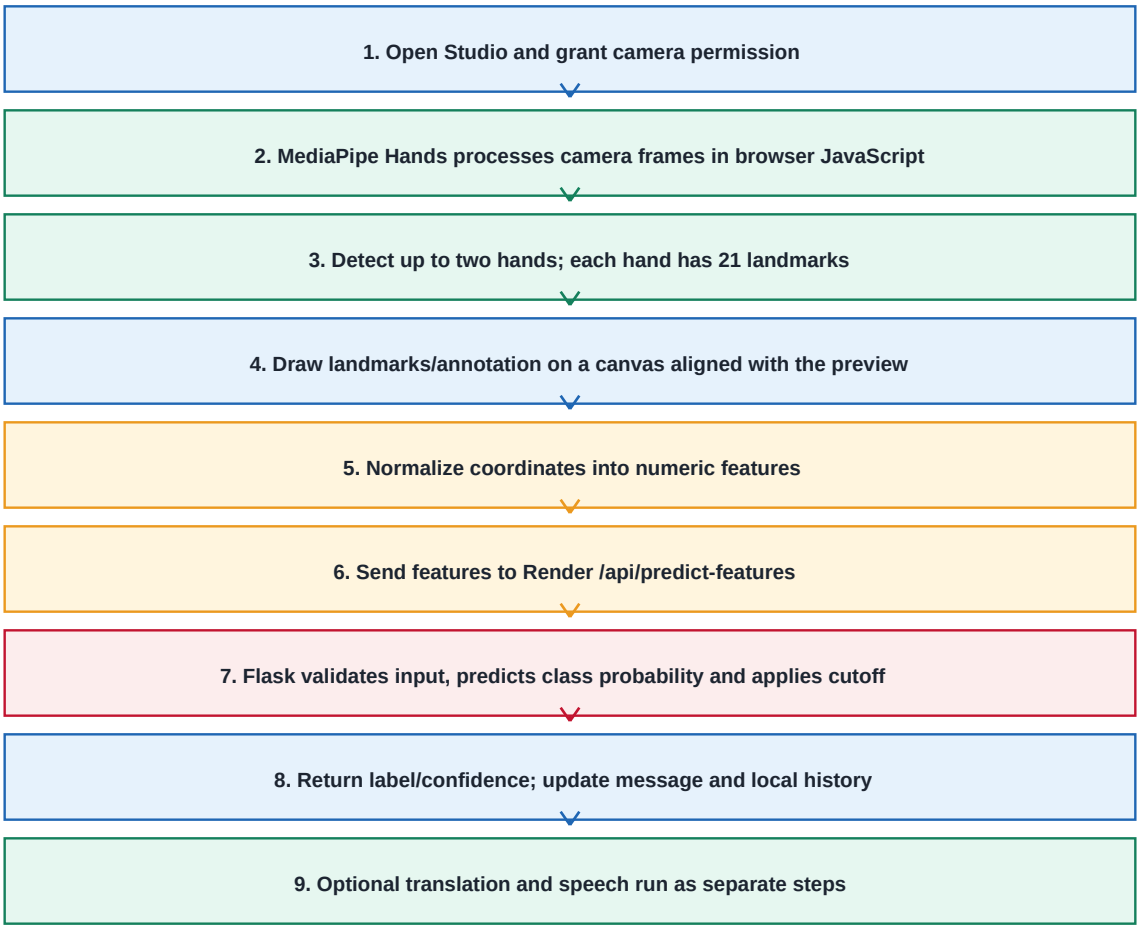
In current Live Studio, MediaPipe tracking runs in the browser. Numeric feature vectors are sent to the Flask classifier route.



Hosting

- Vercel serves the static frontend and deploys connected GitHub commits.
- Render serves the Python Flask backend and loads model artifacts.
- Frontend changes normally deploy on Vercel; backend/model changes deploy on Render.
- A legacy /api/predict image endpoint exists; current Studio primarily uses /api/predict-features.

4. Live Studio runtime

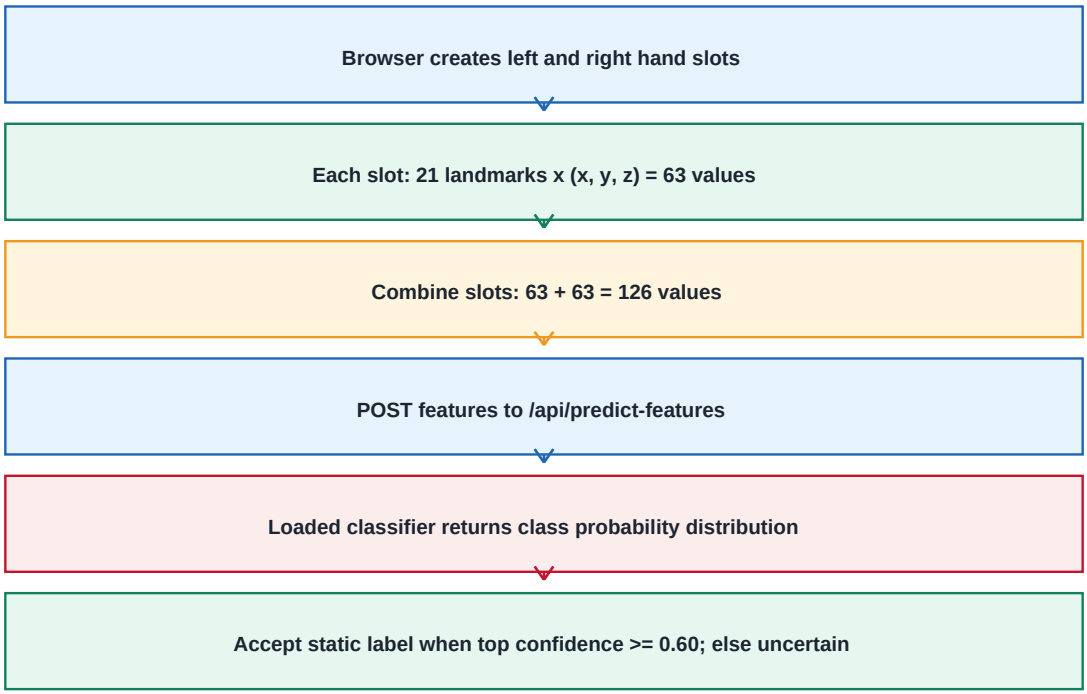


Important distinction

The landmark overlay is a visualization, not the classifier. Recognition works on numeric features. Debug annotation and classification as separate stages.

5. Static-sign recognition

A static sign is represented by a stable pose. The repository label file lists HELLO, I_LOVE_YOU, NO, THANK_YOU and YES; loaded classifier labels from /api/status are authoritative.



Why normalize?

Wrist-relative normalization reduces sensitivity to translation and scale. It cannot eliminate every camera angle, occlusion, lighting or signer-variation issue.

Judge answer

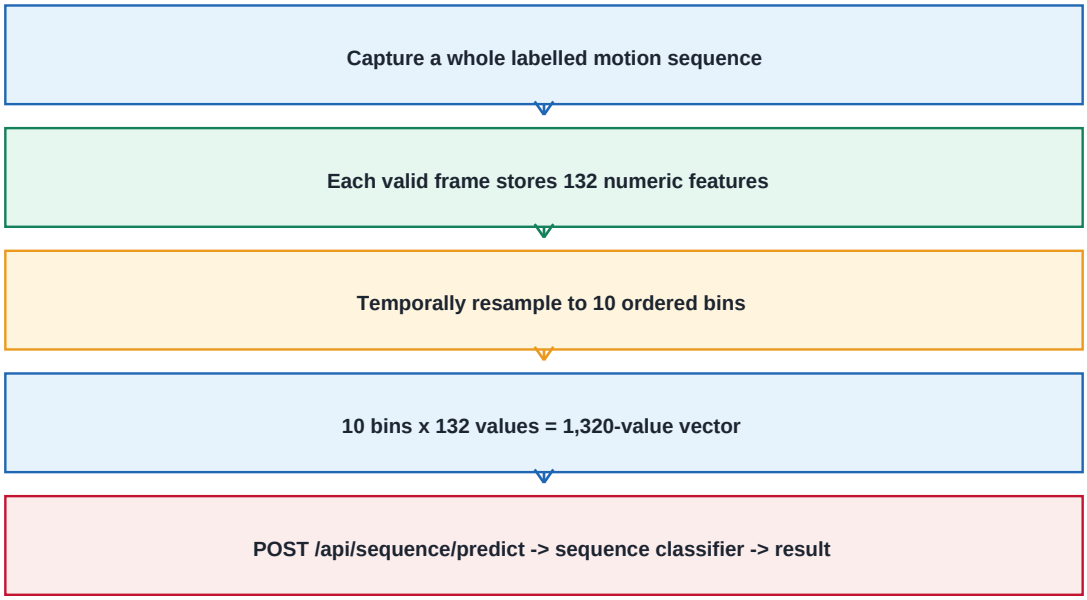
The classifier maps a fixed 126-number vector to a class learned from labelled examples. Adding a label to JSON alone does not make the model learn that class.

6. Continuous/moving-sign recognition

A moving sign depends on pose order over time. SAHAYAK has a separate sequence path and sequence model artifact.

Per-frame representation

The 126 pose values are joined with 6 wrist-position values (x, y, z for both slots), creating 132 values per frame.



Current segmenter settings

- Minimum segment before classifying: 15 frames.
- Maximum retained segment: 90 frames.
- Uses movement and brief stillness (3 still frames) to decide when to send a segment.
- A moving sign needs full ordered examples; a still image cannot teach motion.

7. Feature maths and terminology

Value	Meaning
21 landmarks	Hand tracker output per hand.
3 coordinates	Each point contains x, y and z.
21 x 3 = 63	Values per hand slot.
2 x 63 = 126	Static vector combining left/right slots.
126 + 6 = 132	Continuous frame, adding wrist-position values.
10 x 132 = 1,320	Sequence vector after resampling.
0.60 (60%)	Current acceptance threshold when probabilities exist.
Confidence	Top-class score, not demonstrated real-world accuracy.

Definitions

- Feature: numeric description supplied to a classifier.
- Class/label: category learned during training.
- Training set: examples used to fit a model.
- Test set: examples held aside to estimate performance.
- Confusion matrix: true class versus predicted class counts.
- Overfitting: good training performance, poor generalization.
- Temporal resampling: map variable duration to fixed time points.

Avoid this mistake

A 92% confidence score is not proof that the model is 92% accurate. Accuracy requires a labelled evaluation set separate from training.

8. Backend API map

Route	Purpose
GET /api/health	Checks whether Flask responds; not an accuracy test.
GET /api/status	Model readiness, labels, sequence DB summary, API timings.
POST /api/predict-features	Current static route using 126-value browser-computed features.
POST /api/predict	Legacy image route; server decodes image and runs MediaPipe.
GET /api/sequence/samples	Returns continuous sample counts.
POST /api/sequence/samples	Validates/saves a labelled sequence in SQLite.
GET /api/sequence/export	Exports sequence samples; protect the data.
POST /api/sequence/predict	Runs the continuous classifier.
POST /api/translate	Online translation providers/fallback; language support varies.
POST /api/speech	gTTS speech endpoint; internet/provider needed.

API timing

The backend records successful prediction-route durations in a process-local rolling window up to 200 timings per route. This measures server handling, not total user-perceived response time.

Historical Render logs from 19 October 2026: static API mean 35.3 ms over 50 requests; sequence API mean 113.2 ms over 44 requests. These are historical measurements, not guarantees.

9. Data storage, privacy and offline behavior

Where data lives

- Models: backend/model/sahayak_twohand.joblib and sahayak_sequence.joblib.
- Label metadata: backend/labels_twohand.json; metadata does not train the classifier.
- Sequence samples: backend/continuous_gesture_db.db, SQLite on service filesystem.
- Preferences/custom phrases/history: browser localStorage.
- Code versions: GitHub; Vercel and Render deploy connected commits.

Privacy

- Current Studio sends numeric features to /api/predict-features, not raw video; a legacy image endpoint exists.
- Landmark sequences can be sensitive: obtain consent, limit export and keep identifiable data out of public repos.
- localStorage is not backup and can be cleared.
- SQLite on ephemeral storage may not survive instance replacement; use persistent storage for important records.

Offline

- MediaPipe loads from CDN; the classifier is a remote Render API.
- General translation and gTTS speech use online services.
- A few bundled English/Hindi/Assamese/Bengali phrases may work offline after load; that is not full offline recognition.

10. Verified repository state

Known implementation facts

- Repo: Mrjohanjyotikuli/SAHAYAK-ISL, main branch.
- Frontend: frontend/index.html and frontend/studio.html.
- Backend: Python Flask in backend/web_app.py.
- Requirements include Flask, Flask-CORS, NumPy, OpenCV, scikit-learn, Joblib, MediaPipe, Gunicorn, gTTS.
- Configured labels: HELLO, I_LOVE_YOU, NO, THANK_YOU, YES; /api/status loaded model labels are authoritative.
- Static 126 values; sequence frame 132 values; sequence input 1,320 values.

Important gaps

- Dedicated static webcam capture utility is absent.
- Reusable train_twohand_model.py and train_sequence_model.py scripts are absent.
- No saved held-out accuracy report.
- Sequence DB/API samples alone do not prove those samples trained the deployed classifier.

Say this honestly

The live recognition path is deployed. A repeatable collection, training, evaluation and promotion workflow is still needed before adding signs becomes routine.

11. Add a new STATIC sign

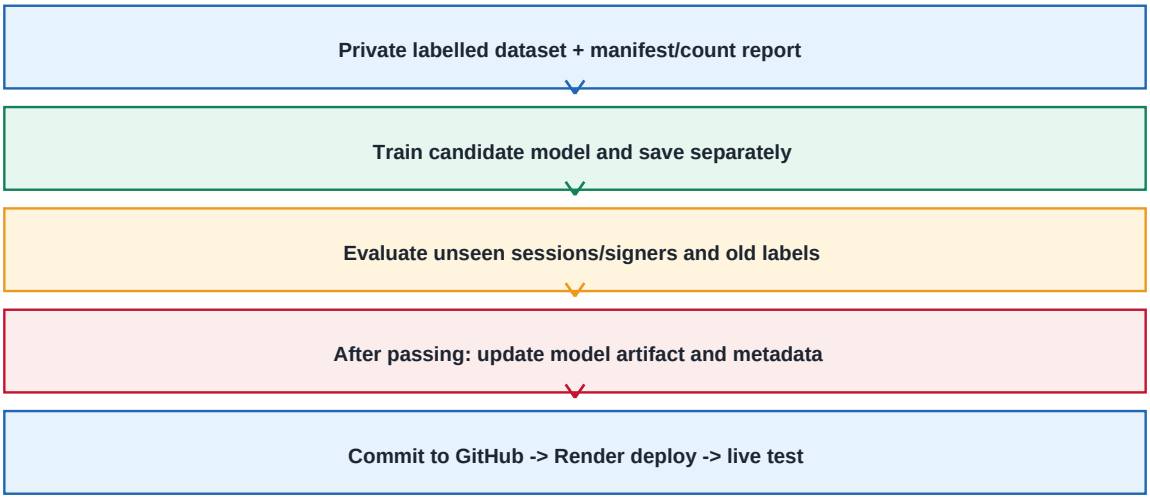
A. Define and collect

- Choose a unique label (example HELP) and define exactly what pose it represents.
- Use a capture tool to collect normalized 126-value rows and pair each row with a label.
- Collect repeated takes with variation in signer, lighting, distance, background and camera angle.
- Include old classes in the training set or the model may forget existing signs.
- Reject missing hands, invalid values and wrong labels; keep private data private.

B. Train and evaluate

- Use scikit-learn with the same feature order/normalization as runtime.
- Split by session or signer before training, not adjacent frames from one burst.
- Report counts, confusion matrix, precision, recall, F1 and held-out accuracy.
- Compare candidate against a fixed regression set of existing classes.

C. Promote safely



12. Add a new CONTINUOUS sign

A moving sign depends on pose order over time. The project stores sequences of 132-feature frames and resamples to a 1,320-value input.

Collect

- Define the label and a consistent start/end boundary.
- Record full ordered performances; include varied repeated takes.
- Store labelled feature frames and session metadata without exposing identities.
- POST /api/sequence/samples exists, but the visible Studio collector is hidden and SQLite is not a durable shared dataset service.

Train and publish

- Create train_sequence_model.py (missing) to load DB/export, call summarize_sequence(), fit classifier and output metrics.
- Split by signer/session; review confusion between similar movements.
- Test segmenter settings: minimum 15 frames, max 90 frames, brief stillness detector.
- Promote only after held-out evaluation and regression tests.

Current blocker

The repository has sequence features, database and inference route, but no reusable sequence-training script. Do not say end-to-end browser training already works.

13. Can collection and training happen in GitHub?

Method	Collect webcam?	Train?	Best use
GitHub editor	No built-in webcam UI.	No; editing labels is not training.	Edit code/docs.
Upload files	Only files collected elsewhere.	Not automatically.	Upload approved data/artifacts.
Codespaces	Only with a designed/tested browser capture workflow.	Yes after scripts/dependencies exist.	Cloud development.
GitHub Actions	Not interactive camera collection by default.	Yes once workflow/data/trainer exist.	Repeatable CI training.
Local .venv	Yes, with capture tool.	Yes once scripts exist.	Best first step for current repo.

What Actions needs

- Workflow under .github/workflows/ plus tested capture, training and evaluation scripts.
- Secure data source; don't put identifiable recordings in public repository.
- Train candidate, print counts/metrics, run regression checks, store candidate artifact.
- Use a promotion gate so failed candidates cannot overwrite the current model.
- After approval, publish artifact, trigger Render and check /api/status plus live recognition.

Recommended path

First make a local Dataset Lab and trainers work, then automate the same checks with GitHub Actions. The normal GitHub editor cannot collect webcam data by itself.

14. Change website content without retraining

Task	File / location	Hosting
Homepage text/layout	frontend/index.html	Vercel
Live Studio UI	frontend/studio.html	Vercel
Logo	frontend/assets/ + HTML path	Vercel
Names/roles/photos	MODULE 10 in index.html; upload consented assets	Vercel
Phrasebook cards	phrasesData in index.html	Vercel
Recognized signs	Train appropriate model + metadata	Render
Backend APIs	backend/web_app.py + requirements.txt	Render
Model artifact	backend/model/*.joblib after testing	Render

Logo and member edits

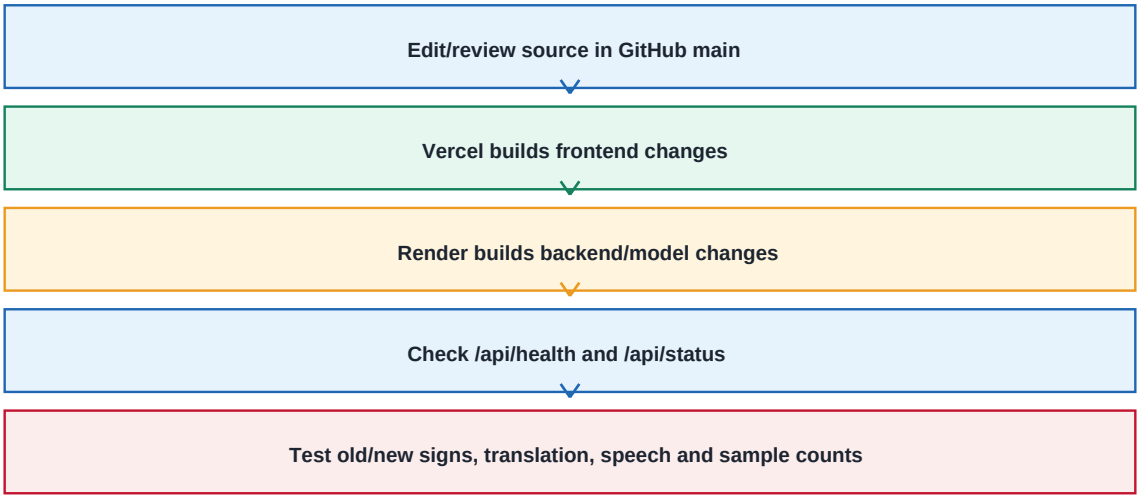
- If replacing only the small emblem, preserve the SAHAYAK wordmark/subtitle.
- Check asset name/case/path. Uploading a PNG does not replace embedded SVG unless HTML points to it.
- Commit to main; wait for Vercel success; refresh phone after cache change.

Keep counts separate

- Phrasebook card count is not the model class count.
- Loaded classifier classes indicate what the camera model can predict.
- Training count must come from the labelled dataset, not labels JSON.

15. Code map and deployment pipeline

File/folder	Responsibility
frontend/index.html	Homepage, menu, phrasebook, demographic counters, team and dataset UI.
frontend/studio.html	Camera, MediaPipe JS, landmarks, static calls, motion segmentation, translation/speech.
frontend/config.js	Render API base URL.
frontend/service-worker.js	Static shell cache; bump version on frontend update.
backend/web_app.py	Flask routes, validation, model loading, prediction, translation/speech.
backend/features.py	126 static values and 132 sequence-frame values.
backend/sequence_features.py	Resample variable sequence length to 1,320 vector.
backend/sequence_db.py	SQLite sample add/load/summary helpers.
backend/labels_twohand.json	Label metadata; not a trainer.
backend/model/*.joblib	Serialized classifiers loaded by backend.
backend/requirements.txt	Python dependencies.



16. Test plan and evidence

Test	Procedure	Evidence
Camera	Start/stop, switch cameras and grant permission.	Video + status.
Static signs	Repeat each active class; compare actual/predicted label.	Outcomes for every class.
Unknown pose	Show an untrained pose.	Inspect uncertainty behavior.
Motion	Perform a supported moving sign.	Actual label vs prediction.
Overlay	Move hand across frame.	Points align with hand.
Translation	Translate supported phrase.	Languages/provider status.
Speech	Speak original/translated text.	Audio or device limitations.
Offline	Test phrase fallback and recognition separately.	Bundled text may work; full recognition may not.
Performance	Measure server and end-to-end separately.	Counts, mean/min/max and conditions.

Evaluation essentials

- Split by signer/session, not adjacent frames.
- Report class counts, precision, recall, F1 and confusion matrix.
- Test occlusion, distance, backgrounds, lighting and new signers.
- Keep fixed regression set for old classes before/after a new sign.
- Record dataset version, model hash, test date, device/browser and protocol.

If asked for accuracy

This repository does not include a verified held-out report, so we do not claim a percentage. The next step is a labelled test set split by signer/session and per-class metrics.

17. Presentation script (4-6 minutes)

Opening

Good morning respected judges. Our project is SAHAYAK, a real-time Indian Sign Language recognition prototype. It demonstrates how a browser camera and hand-landmark features can support communication by recognizing a limited set of trained signs, showing labels and optionally speaking text. It is not a complete ISL translator; we will explain how it works and its limitations.

Demo order

- Show homepage and explain demographic sources versus live project-specific counts.
- Open three-dot menu -> Live Studio.
- Grant camera permission and show landmarks; MediaPipe runs in browser.
- Perform supported signs. Explain confidence is not test accuracy.
- Show moving sign only if sequence model is ready; explain 132/frame and 1,320-vector.
- Type a message, translate and speak; explain internet dependencies.
- Open Dataset Specs and show loaded labels, sample counts and API timing definitions.

Closing

SAHAYAK connects browser-based hand tracking, feature extraction, trained classification, text and speech. Next steps are safe data collection, repeatable training, held-out evaluation and durable storage.

Backup

Keep screenshots or a short recording if Wi-Fi or hosting fails; call it recorded evidence, not live recognition.

18. Three-member speaking roles

The website names these three members. The split below is a suggested presentation plan; adjust it to reflect each person's real contribution. Do not claim someone implemented a module unless it is accurate.

Presenter	Suggested focus	Talking points
Shyamantika Pegu	Problem/access context	Need, goals, sources, intended users, limitations.
Barakha Doley	Frontend/demo	Navigation, Live Studio, phrasebook, translation/speech, UX.
Himanghsu Ghirme	Workflow/testing	Camera, landmarks, feature dimensions, API, model, storage.
Johan Jyoti Kuli	Developer/integration	Repository, hosting, model artifacts, training roadmap.

Handoffs

- Speaker 1 explains the need to turn hand movement into machine-readable features.
- Speaker 2 demonstrates website use.
- Speaker 3 explains features, classifier and testing.
- Developer explains deployment and the need to train/evaluate new signs.

Team tip
All members should know the entire workflow; judges can ask any question to any presenter.

Judge Q&A bank (1/3)

Purpose/features/maths.

Q1. What is SAHAYAK?

A prototype recognizing a limited trained set of hand signs from camera landmarks; it can show and speak output.

Q2. What problem does it address?

It demonstrates a low-friction technical aid for interactions where sign-based and spoken/written communication need support.

Q3. Is it a complete ISL translator?

No. It classifies known labels and does not fully interpret grammar, facial expression, spatial information or spontaneous conversation.

Q4. What does real time mean?

The browser processes frames continually and requests predictions; total speed depends on device, network and server.

Q5. What is a landmark?

A tracked key point represented by numeric coordinates.

Q6. How many hand landmarks?

21 per hand in the MediaPipe Hands representation.

Q7. Why normalize coordinates?

To reduce sensitivity to translation and scale, not eliminate all viewpoint/occlusion issues.

Q8. Why 126 static features?

2 hand slots x 21 landmarks x 3 coordinates = 126.

Q9. Why 132 sequence values per frame?

126 pose values plus 6 wrist-position values.

Q10. Why 1,320 sequence features?

Ten resampled time bins x 132 = 1,320.

Judge Q&A bank (2/3)

Model, data collection and evaluation.

Q11. Which algorithm is used?

The backend loads scikit-learn-compatible classifiers with Joblib; inspect artifact to confirm exact estimator because trainer scripts are missing.

Q12. Does adding a label train the model?

No. Train a model on labelled examples and save a new artifact.

Q13. How add a static sign?

Collect 126-feature rows, train, test on held-out sessions/signers, regression-test old labels, then promote.

Q14. How add a moving sign?

Collect 132-value frames in order, resample to 1,320, train/evaluate a separate classifier, then promote.

Q15. How many samples are enough?

No universal number. Use a balanced varied collection and report actual counts and held-out results.

Q16. What is confidence?

Top-class probability if supported, not a guarantee.

Q17. What is the 60% threshold?

Current code accepts static predictions at >=0.60 and sequence when probabilities exist; it is not accuracy.

Q18. How measure accuracy?

Use unseen labelled examples, confusion matrix, accuracy and per-class precision/recall/F1.

Q19. How avoid overfitting?

Split by signer/session, vary conditions, balance labels, test new examples.

Q20. Can GitHub train it?

Yes through configured Codespaces/Actions after scripts and data inputs exist; normal editor cannot record webcam by itself.

Judge Q&A bank (3/3)

Architecture, privacy, deployment and future improvements.

Q21. Where does camera run?

MediaPipe Hands runs in browser; numeric features go to Render classifier.

Q22. Fully offline?

No. CDN tracker, remote classifier, translation and gTTS require internet; some fixed translations may work offline.

Q23. Important routes?

/api/health, /api/status, /api/predict-features, /api/sequence/predict, /api/sequence/samples, /api/sequence/export, /api/translate, /api/speech.

Q24. What does Vercel do?

Hosts the static frontend and deploys GitHub-connected UI changes.

Q25. What does Render do?

Hosts Flask backend and model files.

Q26. Where are sequences stored?

continuous_gesture_db.db via sequence_db.py; ephemeral storage isn't guaranteed durable.

Q27. Where is history?

Browser localStorage.

Q28. Is raw video always sent?

Primary Studio path sends features; legacy /api/predict can accept images.

Q29. Main limitations?

Small label set, no held-out accuracy report, online dependencies, camera sensitivity and missing reusable trainer scripts.

Q30. Next milestone?

Private Dataset Lab, trainers, held-out evaluation, persistent storage and guarded model promotion.

22. Difficult follow-ups and troubleshooting

Can you prove 98% accuracy?

No held-out report is stored, so do not claim it. Use unseen labelled data and per-class results.

Why sometimes wrong?

Similar poses, limited/imbalanced samples, occlusion, lighting, camera angle, segmentation and training coverage can cause errors.

Does 60% confidence mean 60% accuracy?

No. It is a decision threshold on class probability, not test-set accuracy.

What if internet fails?

Remote classification, CDN tracking, general translation and server speech may fail. Some fixed bundled phrases may still work.

How protect training contributors?

Obtain consent, restrict capture/export, avoid public identifiable data, limit retention, secure APIs and use durable storage.

Troubleshooting table

Symptom	Checks
Camera fails	Permission, HTTPS, camera in use, selected camera and browser support.
No label	/api/status, label set, visible landmarks, feature length, threshold, backend logs.
Overlay misaligned	Video/canvas scale and mirror transform; debug separately from classifier.
New sign not recognized	Phrase card or JSON is not training; collect data and retrain.
Motion sign fails	Sequence model readiness, complete segment, 132 features/frame and class coverage.
Translation fails	Provider/internet error; don't pretend translation succeeded.
Old page on phone	Wait for deploy, refresh/reopen and confirm cache version.
Samples disappear	SQLite may be ephemeral; use persistent disk/DB and backups.

Safe change rule

Never replace the live model until the candidate passes tests for the new sign and every old class. Keep a known-good backup and rollback plan.

23. Glossary, sources and final checklist

Term	Meaning
API	Defined route for requesting data or an action.
Backend	Server code that validates, loads models and responds.
Classifier	Maps feature numbers to one trained label.
Confidence	Top-class score/probability when available.
Dataset	Labelled examples and feature values.
Feature vector	Fixed-length numeric input to a classifier.
Joblib	Python serialization for model objects.
Landmark	Tracked key point on the hand.
MediaPipe Hands	Hand-tracking library used by browser and legacy image route.
Overfitting	Fits training data, generalizes poorly.
SQLite	File-based database used for sequence samples.
Temporal resampling	Map variable sequence duration to fixed time points.
Vercel/Render	Frontend/backend hosting.

Primary sources

Census of India 2011, Table C-20: https://censusindia.gov.in/nada/index.php/catalog/43369/
WHO, Deafness and hearing loss (3 Mar 2026): https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss
SAHAYAK repository: https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL
Live Studio: https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL/blob/main/SAHAYAK-Vercel-Deployment/frontend/studio.html
Flask backend: https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL/blob/main/SAHAYAK-Vercel-Deployment/backend/web_app.py

Last-minute checklist

- Charge laptop/phone; test camera permission; prepare internet/hotspot backup.
- Test every supported class; demo a moving sign only if sequence model is ready.
- Never guess accuracy or dataset size; show live Dataset Specs and explain metrics.
- All team members should know the full pipeline and limitations.

Final message

A strong science project explains what works, how it works, how it was tested, its limits and the evidence needed before expanding its claims.